Binary Trees

> You must understand, young
> Hobbit, it takes a long time to
> say anything in Old Entish.
> And we never say anything
> unless it is worth taking a long
> time to say.
> —Treebeard, Lord of the Rings

Ah, binary trees. These are one of the most *fundamental* data structures in computer science. Most, if not all, popular implementations of databases and file systems make use of some sort of tree to store data efficiently.

We call these structures *trees* because of how we represent them. When we draw binary trees, they often resemble a tree (albeit upside-down).

For most computer science applications, binary trees are **rooted**. That is, a special node `r` is called the *root* of the tree. Much like the *sentinel* in a skiplist. The **degree** of a node `u` is the number of *edges* or *connections* that originate from `u`. In a singly-linked list, for example, the degree of any node in the list is 1. Since we only have one edge going out of the node: `next`. The *root* of the tree has a degree of at most 2. In any binary tree node `u`, we call the outgoing links `left` and `right` child; together, they are called the children of `u`. Additionally, in a binary tree node $\texttt{u} \neq \texttt{r}$, we also have a link to the **parent** of the node (the *root* has no parent); hence `u` has degree 3. We illustrate a small binary tree in fig. 23.

Terminology dump! The **depth** of a node `u` in a binary tree is the length of the path from `u` to the *root* of the tree. In other words, this counts the number of edges we have to follow to get from `u` to `r`. In a path from `u` to the root `r`, any node `w` in between is considered an **ancestor** of `u` (the *direct* ancestor of `u` is called the parent) and `u` is considered a **descendant** of `w` (the *direct* descendant of `w` is called its children). The **subtree** of a node `u` is the binary tree rooted at `u` and contains all of `u`s descendants. The **height** of a node, `u`, is the length of the *longest* path from `u` to one of its descendants. The **height** of a tree is the height of its root. Finally, a node with no children is called a **leaf**.

We sometimes think of the tree as being terminated with **external nodes**. Any node that doesn't have a left child has an *external node* as its left child. Same goes if the node doesn't have a right child.

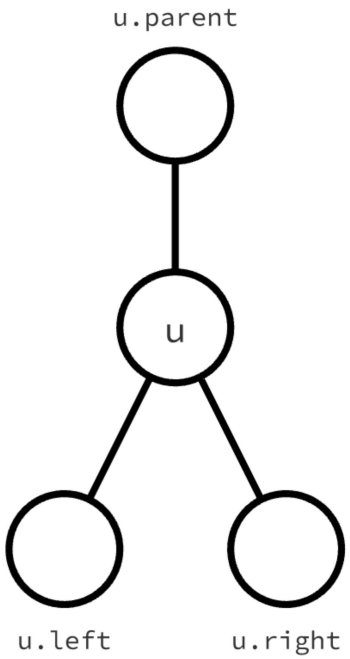


Figure 23: Parent, left child and right child of a node `u` in a `BinaryTree`

BinaryTree: Basic Binary Tree

The *simplest* way to represent a node `u` in a binary tree is store the neighbors of `u` (`u`'s parent, and 2 children).

```
class BTNode {
    BTNode *left;
    BTNode *right;
    BTNode *parent;

    BTNode() {
        left = right = parent = NULL;
    }
}
```

The binary tree itself can then be represented by a pointer to its root node, `r`.

```
class BinaryTree {
    BTNode *r;
}
```

We can compute the `depth()` of a node `u` in a binary tree by *counting* the number of steps on the path from `u` to the root `r`. All we have to do

is count how many times we follow `u.parent` until we reach a `null`, at which point we are already at the root.

Recursive Algorithms

Using a recursive algorithm makes it very easy to *write* code to deal with binary trees. For example, we can count the number of nodes in a tree by using a recursive `size()` function:

```
int size(Node *u) {
    if (u == NULL) {
        return 0;
    }
    return 1 + size(u->left) + size(u->right);
}
```

It's elegant, and makes sense.

We can also compute for the `height()`:

```
int height(Node *u) {
    if (u == NULL) {
        return -1;
    }
    return 1 + max(height(u->left),
     ↪  height(u->right));
}
```

The height of a node `u` is defined as 1 + the height of the higher child. Isn't that pretty?

Traversals

We can *traverse* binary trees recursively as well:

```
void traverse(Node *u) {
    if (u == NULL) {
        return;
    }
    traverse(u->left);
    traverse(u->right);
}
```

The above code does nothing *but* to visit all the nodes under the `u` subtree. The order in which we call `traverse()` actually determines the order in which we traverse the tree. Consider the tree in fig. 24, we have 4 types of traversals:

1. In-order: visits the left child first, then itself, then the right. (0, 1, 2, 3, 4, 5). See fig. 25.
2. Pre-order: visits itself first, then the left, then the right. (3, 1, 0, 2, 5, 4)
3. Post-order: visits the left child first, then the right, and only then itself. (0, 2, 1, 4, 5, 3)
4. Level-order: visits nodes level-by-level (same depth, going left to right like reading a book), starting at the root. (3, 1, 5, 0, 2, 4)

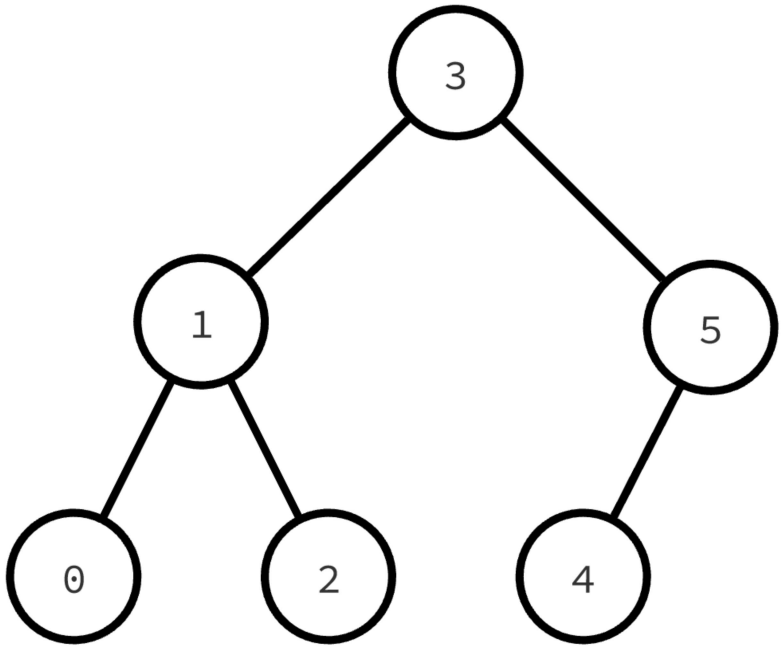


Figure 24: A sample tree of n = 6

Below we have the code for each of the type of traversals visiting all the nodes.

```
void inorder(Node *u) {
    if (u == NULL) {
        return;
    }
    inorder(u->left);
    cout << "visit!"
    inorder(u->right);
}
```

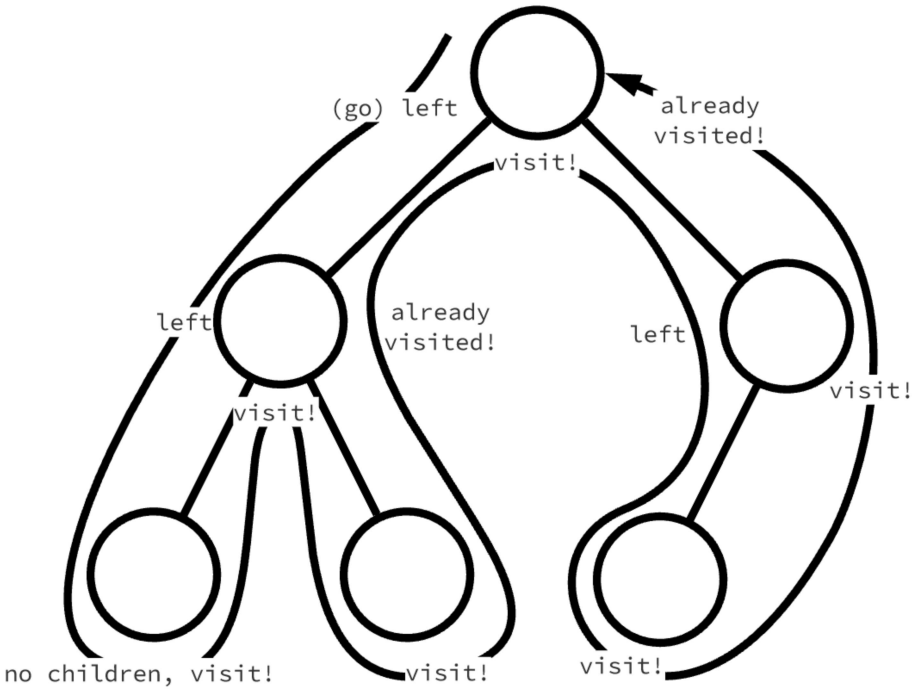


Figure 25: A in-order traversal flow

```
void preorder(Node *u) {
    if (u == NULL) {
        return;
    }
    cout << "visit!"
    preorder(u->left);
    preorder(u->right);
}
```

```
void postorder(Node *u) {
    if (u == NULL) {
        return;
    }
    postorder(u->left);
    postorder(u->right);
    cout << "visit!"
}
```

```
void levelorder(Node *u) {
    ArrayDeque<Node*> q;
    if (u != NULL) {
        q.add(0, u);
    }
    while (q.size() > 0) {
        Node *w = q.remove(q.size() - 1);
        cout << "visit!"
        if (w->left != NULL) {
            q.add(0, w->left);
        }
        if (w->right != NULL) {
            q.add(0, w->right);
        }
    }
}
```

The `levelorder()` method is unlike the others. We had to use a Queue `q` that starts with the starting node, then we add its children to the queue to process later on. The order in which we push and pop nodes via the queue implements the level-order traversal that we want.

Recursion Caveat

Recursion *could* be problematic. If the height of the tree is *very* large, recursion could cause a crash. This is because recursion consumes a lot of stack space, depending on how many times we make recursive call. Overflowing this stack space will cause a crash. We can also traverse a tree without recursion. We just need to keep track of *where* we came from; this will determine where we go next in the tree.

For instance, in an in-order traversal, if we arrive at a node `w` from `w.parent`, then we go to `w.left`. If we came from `w.left`, then we mark `w` as visited, and go to `w.right`. If we came from `w.right`, then we are done with `w`'s subtree and we jump back up via `w.parent`.

```
void traverse2(BTNode *u) {
    BTNode *w = u;
    BTNode *prev = NULL;
    BTNode *next = NULL;

    while (w != NULL) {
        if (prev == w->parent) {
            if (w->left != NULL) {
                next = w->left;
            } else if (w->right != NULL) {
                next = w->right;
            } else {
                cout << "visit!";
                next = w->parent;
            }
        } else if (prev == w->left) {
            cout << "visit!";
            if (w->right != NULL) {
                next = w->right;
            } else {
                next = w->parent;
            }
        } else {
            cout << "visit!";
            next = w->parent;
        }
        prev = w;
        w = next;
    }
}
```

Similarly, we can calculate the size of the tree non-recursively

```
int size2(BTNode *u) {
    BTNode *w = u;
    BTNode *prev = NULL;
    BTNode *next = NULL;

    int n = 0;
    while (w != NULL) {
        if (prev == w->parent) {
            n++;
            if (w->left != NULL) {
                next = w->left;
            } else if (w->right != NULL) {
                next = w->right;
            } else {
                cout << "visit!";
                next = w->parent;
            }
        } else if (prev == w->left) {
            cout << "visit!";
            if (w->right != NULL) {
                next = w->right;
            } else {
                next = w->parent;
            }
        } else {
            cout << "visit!";
            next = w->parent;
        }
        prev = w;
        w = next;
    }
}
```

Now that we know how a tree looks like, and how it's implemented, let's learn how to *use* it.

BinarySearchTree

A `BinarySearchTree` is a special kind of binary tree in which each node `u` also stores a value `x` from some total order (based on some order). The values in a binary search tree *obey* the **binary search tree**

property. That is, for a node `u`, `u.left` contains values *less* than `u.x`. Conversely, nodes under `u.right` contain values *greater* than `u.x`. A sample binary search tree is illustrated in fig. 26.

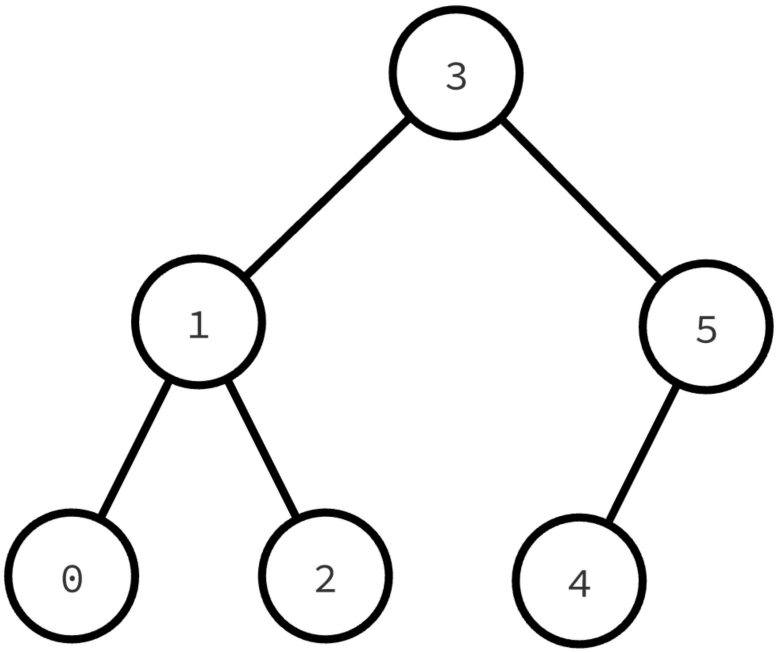


Figure 26: A sample binary search tree

`find(x)`

The binary search tree property is useful here because it helps us quickly locate a value in the tree. To do this, we start from the root, `r`. Then when we examine a node, `u`, there are 3 cases:

1. If `x` $<$ `u.x`, then we set `u` to `u.left` since smaller numbers are located in the left child.
2. If `x` > `u.x`, then we set `u` to `u.right` since larger numbers are located in the right child.
3. If `x` = `u.x`, then we've found the node `u` containing `x`.

We stop searching when the 3rd case is reached, or when `u` is `null`. If the 3rd case is reached, we simply return `u.x`. Otherwise, when `u` is `null`, we conclude that `x` is not in the tree; however, we can still return the smallest value that is $\geq$ `x`. This is because the last node before we get a `null` actually holds the closest value to `x`. In the 1st case above, the previous node would hold the smallest value that is *greater* than `x`. Similarly, if the 2nd case occurred, the previous node before getting a `null` would hold the largest value in the tree that is *less* than `x`. We can keep track of the last node when the 1st case occurs. Conveniently, this would help us implement the `find(x)` operation that returns the smallest value stored in the tree that is $\geq$ `x`.

```
T find(T x) {
    BTNode *w = r;
    BTNode *z = NULL;
    while (w != NULL) {
        int comp = compare(x, w->x);
        if (comp < 0) {
            z = w;
            w = w->left;
        } else if (comp > 0) {
            w = w->right;
        } else {
            return w->x;
        }
    }
    return z == NULL ? NULL : z->x;
}
```

`add(x)`

To add a new value `x` to a `BinarySearchTree` we first have to look for `x`. If we find `x`, there is no need to insert it. Otherwise, we insert it at a leaf child of the last node we reach when searching for `x`. Whether we insert it as a left child or right child depends on whether `x` is lesser or greater than the value of that last node. We illustrate the process in fig. 27.

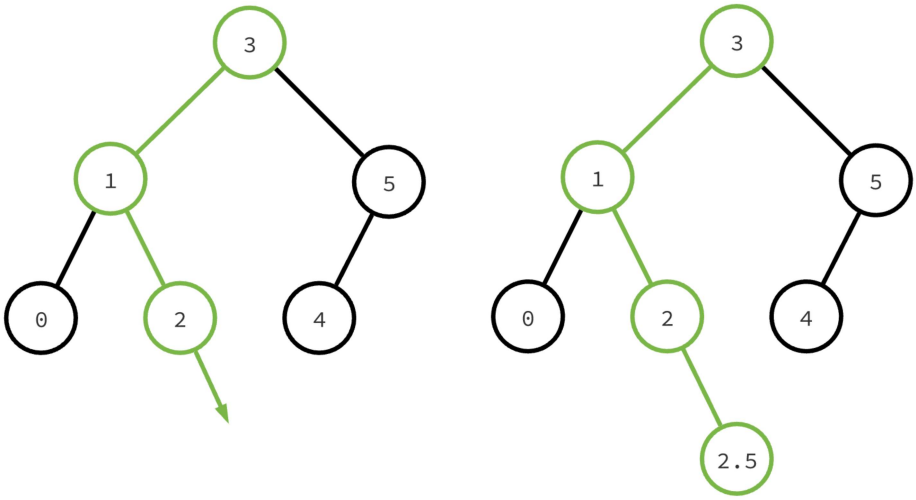


Figure 27: Inserting 2.5 into a `BinarySearchTree`

`remove(x)`

Removing a node is a little more difficult than adding one in. If `u` is a leaf, we can simply remove it from the tree by removing it from `u`'s parent. If `u` has one child, we let `u`'s parent adopt its child; we can splice `u` from the tree—similar to splicing a node from a linked list.

```
void splice(BTNode *u) {
    BTNode *s;
    BTNode *p;
    if (u->left != NULL) {
        s = u->left;
    } else {
        s = u->right;
    }
    if (u == r) {
        r = s;
        p = NULL;
    } else {
         p = u->parent;
         if (p->left == u) {
            p->left = s;
         } else {
            p->right = s;
         }
    }
    if (s != NULL) {
        s->parent = p;
    }
    n--;
}
```

However, when `u` has 2 children, things get a little trickier. The simplest thing we can do is to find a node `w` in the tree to replace `u`. The node `w` must maintain the binary search tree property and so must have a value close to `u`. For instance, a node `w` that is the *smallest* value *greater* than `u` will work for our replacement. We can find `w` by taking the smallest node value in the subtree rooted at `u.right`. Conversely, we can use the node with the largest value in the subtree rooted at `u.left`. To replace `u`, we remove `w` from the tree (which should be easy since it has no children) and replace the value of `u` with the one in `w`. We illustrate this in fig. 28.

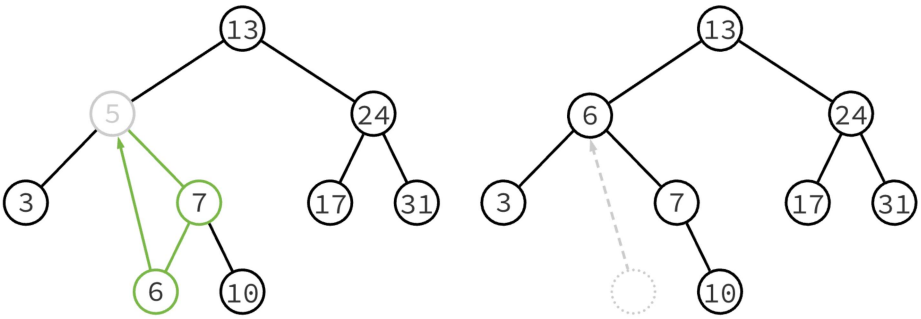


Figure 28: Deleting a value (5) with 2 children from a binary search tree, replacement value path is highlighted in green

```
void remove(Node *u) {
    if (u->left == NULL || u->right == NULL) {
     ↪  // means u has 2 children
        splice(u);
        delete u;
    } else {
        Node *w = u->right;
        while (w->left != NULL) {
            w = w->left;
        }
        u->x - w->x;
        splice(w);
        delete w;
    }
}
```

Analysis

If you notice, all our operations in the `BinarySearchTree` involve *following* a path from the *root* of the tree to some node in the tree. Without having a required structure for the `BinarySearchTree`, we can't say much about the length of this path; except for us to say that the length of the path is less than the number of nodes in the tree. Since all of these operations are limited by the length of the path, we can only say that all these operations run in $O(n)$ time.

$O(n)$? That's *really* unimpressive. Why? This is because a `BinarySearchTree`, as it is, can become **unbalanced**. Instead of looking like a tree in some figures we've seen (fig. 26), it could possibly look like a long chain of nodes, just like a linked list (see fig. 29). We call a tree **size-balanced** if for every node `u`, the size of the subtrees rooted at `u.left` and

`u.right` differ by at *most* 1. In fig. 29, it is clearly unbalanced, because the height of the right child is 0.

We summarize the operations in tbl. 10.

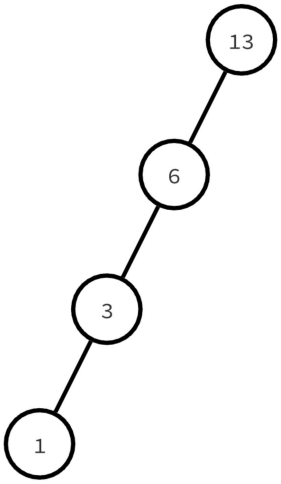


Figure 29: A heavily unbalanced `BinarySearchTree`

Table 10: Summarized time complexity for a `BinarySearchTree`

Operation	Time Complexity
`add(x)`	$O(n)$
`remove(x)`	$O(n)$
`find(x)`	$O(n)$